

ที่มาและความสำคัญของโครงงาน

- ปัจจุบันเกมแนวแก้ปริศนา (Puzzle) รูปแบบดินกล่อง (Sokoban) มักมีข้อจำกัดด้านพื้นที่แบบ 2 มิติแบบราบ โครงงานนี้จึงนำเสนอ "Solar Paradox" เกมที่ฉีกกฎเกณฑ์เดิมๆ ด้วยการนำกลไก "โลกซ้อนโลก (Recursive & Sub-world Mechanics)" มาเป็นแกนหลัก
- ผู้เล่นจะไม่ได้ทำเพียงแค่การเคลื่อนย้ายวัตถุ แต่ต้องใช้จินตนาการเพื่อ "หลุดเข้าไปในกล่อง" ซึ่งเปรียบเสมือนประตูมิติสู่ฉากย่อยที่ซ้อนทับกันอยู่ เกมนี้จึงช่วยฝึกทักษะการคิดวิเคราะห์เชิงตรรกะ (Logical Thinking) และการวางแผนจัดการพื้นที่ที่เชิงซ้อน (Spatial Awareness) ภายใต้ธีมการสำรวจอวกาศสุดล้ำ

ขอบเขต เป้าหมาย และข้อกำหนด

**เป้าหมายหลัก:** พัฒนาตัวต้นแบบ (Prototype) ของเกม 2D Puzzle ที่สมบูรณ์แบบสามารถสาธิตระบบการเดินทางข้ามมิติระหว่างโลกหลักและโลกย่อยได้อย่างไร้รอยต่อ

**ขอบเขตของระบบ :**

- Grid-Based Movement Controller:** ระบบควบคุมตัวละครและการดันวัตถุที่อ้างอิงตามช่องตาราง (Grid) อย่างแม่นยำ
- Interactive Entities:** ระบบวัตถุที่ตอบสนองได้หลากหลาย เช่น สวิตช์แรงกด (Pressure Plates) และจุดผ่านด่าน (Goal Points)
- Recursive Transition:** ระบบการสลับฉากและย้ายพิกัดตัวละครเข้า-ออกกล่องมิติย่อย พร้อมการจัดวางหน้าต่าง UI ที่สอดคล้องกัน

**ข้อกำหนด:** พัฒนาเป็นแอปพลิเคชันสำหรับแพลตฟอร์ม PC ควบคุมผ่านระบบ Keyboard และนำเสนอด้วยงานภาพสไตล์ 2D Pixel Art

การออกแบบหรือแนวทางเชิงเทคนิค

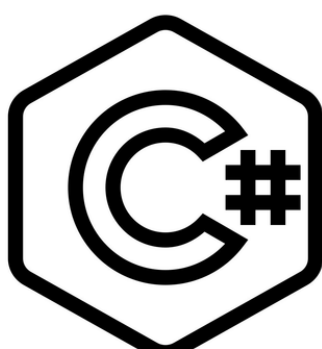
- ตรรกะโลกซ้อนโลก (Recursive Worlds):** ออกแบบระบบให้กล่องเป้าหมายเป็น "ประตูมิติ" (Gateway) ผู้เล่นสามารถเดินหลุดเข้าไปในกล่องเพื่อไขปริศนาในด้านย่อย (Sub-world) ที่ซ้อนอยู่ภายในได้
- ระบบกล่องซ้อนกล่อง (Box-in-Box):** เพิ่มความซับซ้อนด้วยระบบที่อนุญาตให้ "ดินกล่องในหนึ่ง เข้าไปในกล่องอีกในหนึ่ง" ได้ โดยระบบจะคำนวณเงื่อนไขและย้ายพิกัดกล่องที่ถูกดันให้ไปปรากฏในมิติย่อยอย่างแม่นยำ
- การข้ามมิติไร้รอยต่อ (Seamless Transition):** เมื่อมีการข้ามมิติ ระบบจะทำงาน 2 ส่วนคือ ย้ายพิกัดแบบทันที (Coordinate Teleportation) และสลับมุมมองกล่อง (Virtual Camera) ร่วมกับการจัดการ Layer เพื่อสร้างภาพลวงตาของการซูมเข้า-ออก
- การจัดการการชนเชิงพื้นที่ (Spatial Collision):** ประยุกต์ใช้ Raycast 2D ยังเส้นตรวจสอบพื้นที่ล่วงหน้า ป้องกันบั๊กการดันกล่องทะลุกัน หรือดันเข้ามิติย่อยในขณะที่ปลายทางมีวัตถุขวางอยู่

เครื่องมือ เทคโนโลยี และทรัพยากรที่ใช้

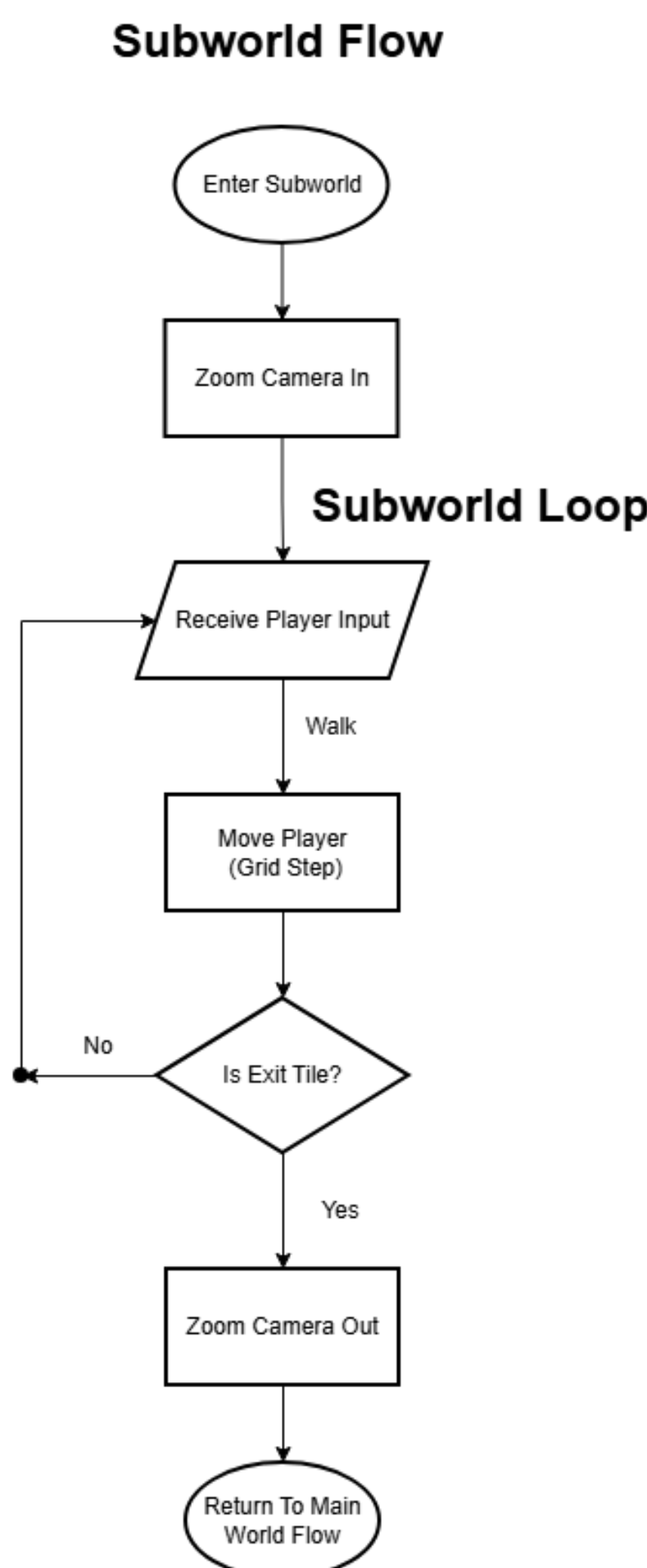
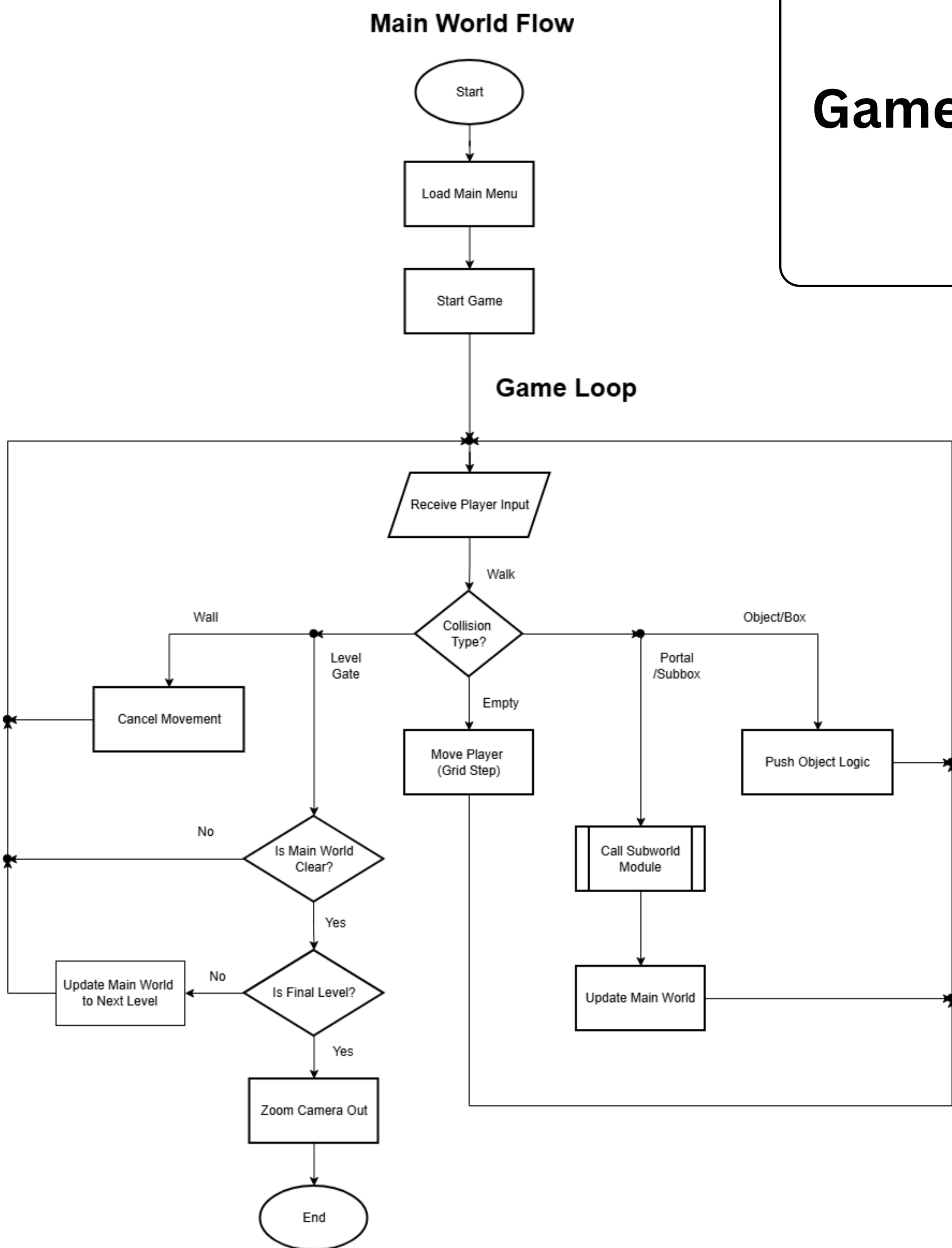
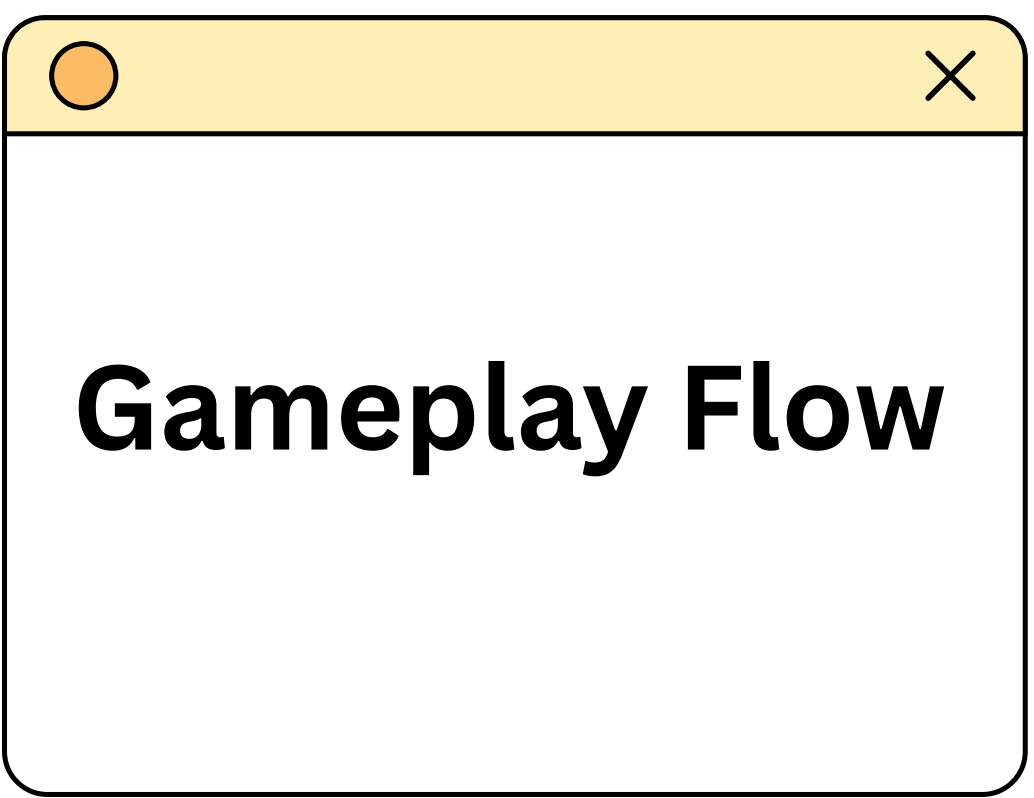
- Game Engine:** พัฒนาด้วย Unity 6 ซึ่งมีประสิทธิภาพสูงในการจัดการ 2D Physics และ 2D Render Pipeline
- Programming Language:** เขียนสคริปต์ควบคุมโครงสร้างลอจิกด้วยภาษา C# (Object-Oriented Programming)
- Graphics Tools:** ใช้ Unity Sprite Editor ในการตัดแบ่งชิ้นส่วนภาพ (Sprite Slicing) และจัดวางฉากด้วย Tilemap System
- Asset Credits:** ทรัพยากรภาพ 2D ได้รับการสนับสนุนสิทธิ์การใช้งานจากครีเอเตอร์อิสระบนแพลตฟอร์ม itch.io และ Unity Asset Store



Unity



itch.io



การทดสอบหรือการประเมินผล

- Unit Testing:** ตรวจสอบความแม่นยำของระบบตาราง ยืนยันว่าตัวละครและวัตถุจะจัดวางอยู่กึ่งกลางช่องตารางเสมอ (Snap to Grid) เมื่อหยุดเคลื่อนที่
- Edge Case Testing:** ทดสอบกรณีขอบเขตที่ซับซ้อน เช่น การดันกล่องซ้อนกันหลายใบ หรือการดันวัตถุทะลุมิติพร้อมกัน เพื่อทดสอบความเสถียรของ Raycast
- Gameplay & UX Testing:** ให้ผู้เล่นทดลองหลุดเข้า-ออกกล่องมิติย่อย เพื่อประเมินความสับสนของมุมมองกล่อง และให้แน่ใจว่าปริศนาสามารถแก้ได้จริง (No Soft-lock)